Module 3: Prompt Engineering

Prompt Engineering

The art and science of communicating with LLMs effectively

Mastering the Art of Effective Prompting

Today's Journey

01

Anatomy of a Great Prompt

Understanding the essential building blocks

02

Zero-Shot vs Few-Shot

When to teach by telling vs showing

03

Chain-of-Thought Reasoning

Making models think step by step

04

ReAct Pattern for Agents

The foundation of modern AI agents

What You'll Master

01

Structured Outputs

Getting reliable, parseable JSON responses

02

System Prompts

Shaping agent behavior from the foundation

03

Prompt Templates

Best practices for production systems

Duration: 60 minutes | **Slides:** 14

Anatomy of a Great Prompt

Every effective prompt has five essential components that work together to guide the LLM toward reliable, accurate responses.

Context

Who are you? What's the situation? This primes the model to respond appropriately.

Instruction

What specific task should be done? Be explicit and precise about the action required.

Input Data

What information to work with? Clearly delineate this from instructions.

Output Format

How should the response be structured? Don't leave the model guessing.

Constraints

What rules or limitations apply? What shouldn't the model do?

Example: "You are a customer service agent for TechCorp. Analyze the following complaint and categorize it. Input: 'My laptop won't turn on after the latest update...' Respond with JSON containing: category, severity, suggested_action. Do not promise refunds."

Zero-Shot vs Few-Shot Prompting

Understanding when to teach by telling versus teaching by example is fundamental to effective prompt engineering.

Zero-Shot Prompting

No examples provided—the model relies on its general knowledge from training.

```
Prompt: "Classify this text as
positive, negative, or neutral:
'The product arrived damaged but
customer service was helpful.'"
```

The model infers what you mean by classification based on patterns learned during training.

Best For:

- Simple, well-understood tasks
- Token efficiency matters
- Model already performs well

Few-Shot Prompting

Examples demonstrate the desired behavior pattern clearly.

```
Prompt: "Classify text sentiment.
Examples:
Text: 'I love this product!'
Sentiment: positive

Text: 'Worst purchase ever.'
Sentiment: negative

Text: 'The product arrived damaged...'
Sentiment: ???"
```

Best For:

- Complex or nuanced tasks
- Specific output format required
- Domain-specific patterns
- Model struggles zero-shot

Chain-of-Thought Prompting

Making the model "think out loud" dramatically improves accuracy on complex, multi-step reasoning tasks.

The Problem

LLMs often fail at multi-step reasoning when asked for direct answers. They pattern-match to familiar responses, which may be incorrect.

Question: "What is 15% of the total if the subtotal is $85 and there's a $10 discount?"

❌ Direct answer: "$12.75" (wrong)

The Solution

Ask the model to show reasoning step by step. Each step becomes context for the next.

Chain-of-Thought approach:

1. Start with subtotal: $85
2. Apply discount: $85 - $10 = $75
3. Calculate 15%: $75 × 0.15 = $11.25

✓ Answer: $11.25 (correct)

CoT Techniques

- **"Let's think step by step..."** — Simple trigger
- **"First... Then... Finally..."** — Structured framework
- **"Show your work."** — Makes reasoning visible

Why CoT Works for Agents

- **Interpretability:** See decision-making process
- **Debugging:** Find where reasoning failed
- **Accuracy:** Reduces errors on complex tasks
- **Tool Selection:** Better choices when reasoning first

Advanced Chain-of-Thought Patterns

Sophisticated reasoning techniques that build on basic CoT for even greater reliability and accuracy.

Self-Consistency

Run the same prompt multiple times with temperature > 0, then take the majority answer to reduce random errors.

- Run 1: "$11.25" ✓
- Run 2: "$11.25" ✓
- Run 3: "$12.75" ×
- **Majority: $11.25**

Tree of Thoughts

Explore multiple reasoning paths simultaneously and backtrack when paths fail—more thorough than linear CoT.

Problem → Path A (dead end) → Path B → B1 (solution ✓)

Self-Reflection

Model evaluates its own output: "Did you consider all constraints? Any edge cases? Would you change anything?"

Verification Chain

Separate generation from verification: one prompt generates the solution, a different prompt checks it.

Trade-off: These patterns add latency and cost. Use them when accuracy matters more than speed—for important decisions, financial calculations, or complex planning.

ReAct Pattern for Agents

Reasoning + Acting = ReAct

The foundational pattern underlying most modern AI agents, combining chain-of-thought reasoning with real-world tool use.

Thought

Reason about the current situation and what action to take next.

"I need to find the current stock price..."

Action

Decide what tool to use and how to use it.

`get_stock_price(symbol='AAPL')`

Observation

Receive real data from the tool—grounded in reality, not hallucinated.

Result: Price: $187.50, Change: +1.2%

Why ReAct Works

- **Transparent reasoning** — You see the thought process
- **Grounded in real data** — Tools provide facts, not hallucinations
- **Interruptible** — Can stop and review at any step
- **Debuggable** — Find exactly where reasoning went wrong

Structured Outputs & JSON Mode

Getting reliable, parseable responses is critical for agents. Your code needs structured data to parse and act on, not prose.

1

The Problem

❌ "The sentiment is positive with a score of about 8 out of 10."

Prose is hard to parse reliably. Agents need structured data.

2

Prompt Engineering

"Respond with JSON only. No other text. Schema: {sentiment: string, score: number}"

Works most of the time, but can still output invalid JSON.

3

JSON Mode

`response_format={"type": "json_object"}`

Guarantees valid JSON, but doesn't guarantee schema compliance.

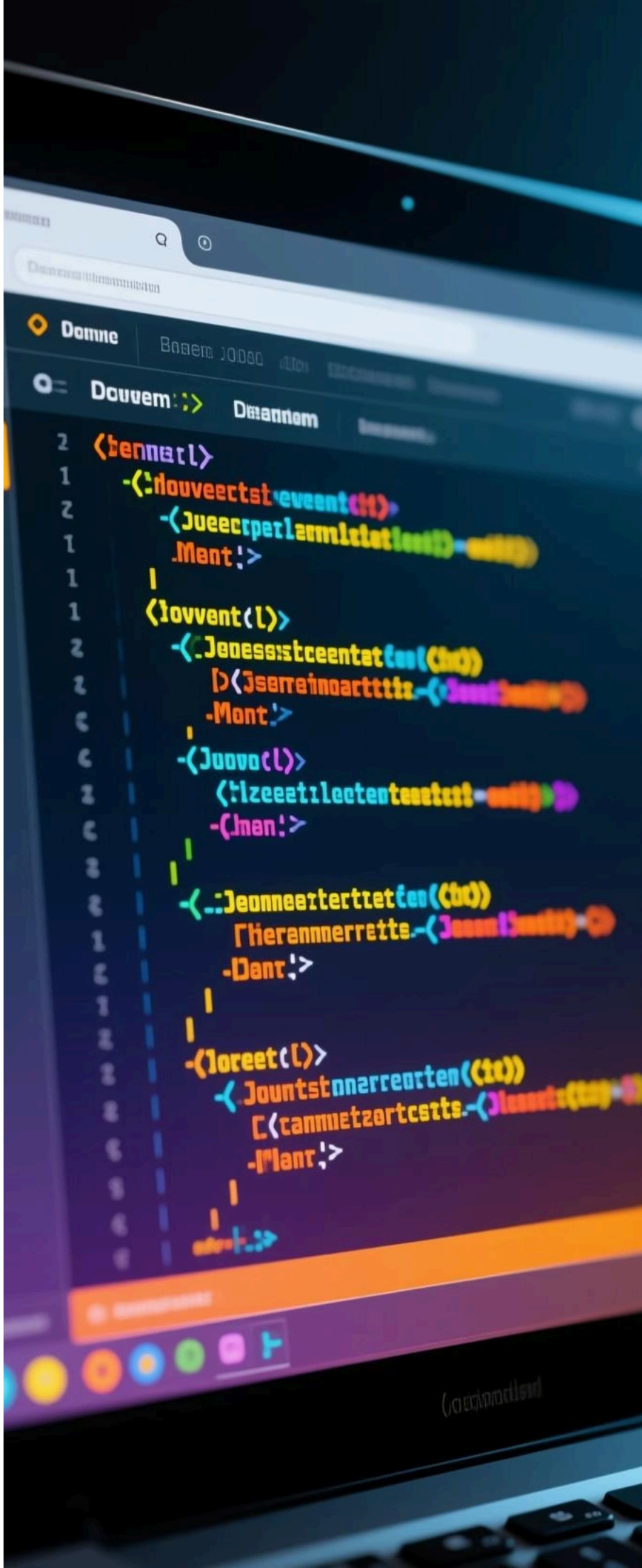
4

Structured Outputs ✓ BEST

`response_format={"type": "json_schema", "schema": {...}}`

Guarantees valid JSON matching your exact schema.

Example output: `{"sentiment": "positive", "score": 0.95}` — Clean, parseable, guaranteed to match your schema structure.



System Prompts for Agents

The foundation that shapes all agent behavior. Everything your agent does is influenced by this critical prompt.

1

Identity

Who is this agent? A name and role help the model adopt appropriate behaviors.

"You are Aria, a customer service agent for TechCorp."

2

Capabilities

List tools explicitly with descriptions of when to use each one.

"You have access to: order_lookup, return_request, faq_search."

3

Behavior Guidelines

What should the agent always do? Never do? How to handle uncertainty?

"Always be helpful and empathetic. Never share personal customer data."

4

Response Format

Should responses be formal or casual? When to use bullet points vs paragraphs?

5

Constraints

Business rules, safety constraints, limitations—guardrails for agent behavior.

"Cannot process refunds over $500. Must verify identity before account changes."

6

Examples (Optional)

Sample interactions showing desired behavior patterns in action.

Prompt Templates: Best Practices

Creating maintainable, effective prompts that scale from prototype to production systems.

Use Clear Delimiters

XML tags, triple quotes, or markdown headers separate instructions from data.

```
<message>{content}</message>
```

Validate Inputs

Sanitize user inputs before inserting into templates. Prevent prompt injection attacks.

Version Your Prompts

Track changes over time. A/B test different versions. Know what's in production.

Test Edge Cases

Empty inputs, very long inputs, special characters, multiple languages.

Iterate Based on Data

Log failures. Analyze patterns. Continuously refine based on real usage.

Document Everything

Explain why each section exists. Future you will thank present you.

> "The difference between a mediocre agent and a great one often comes down to how well the prompts are crafted. Invest time in your prompt engineering—it pays compound returns."